BQN-UY

Equipo de Desarrollo

CI/CD v2

Guía de referencia de actions y workflows

Organización BQN-UY/CI-CD · Marzo 2026

Versión	v2.0
Audiencia	Equipo de Desarrollo
Estado	Borrador — revisión equipo
Clasificación	Confidencial — uso interno

1. Introducción y principios de diseño

Este documento describe la estructura del repositorio `BQN-UY/CI-CD`, que centraliza toda la lógica de CI/CD de la organización. Los repositorios de proyecto no contienen lógica de build propia — solo definen cuándo disparar cada workflow.

1.1 Principios

- **Sin duplicación —** la lógica de build, lint, test y deploy vive una sola vez en CI-CD.
- **Cero fricción para el desarrollador —** una línea `uses:` reemplaza decenas de pasos.
- **Escalabilidad —** agregar un nuevo stack es agregar una carpeta. El resto no se toca.
- **Seguridad por defecto —** OpenGrep y BetterLeaks corren en cada PR sin configuración adicional.
- **SemVer 2.0 estricto —** la versión vive en el tag de Git. No hay archivos de versión que colisionen en back-merge.

1.2 Stacks cubiertos

Capa	Stack	Prefijo de carpeta
Frontend	HTML + JS standalone	frontend/html-js/
Frontend	Vaadin (webapps Java)	frontend/vaadin/
Frontend	Flutter / Dart (apps + web)	frontend/flutter/
Backend	Python 3 + FastAPI	backend/python/
Backend	Scala + Pekko (APIs)	backend/scala/
Backend	Node.js	backend/node/
Compartido	Todos los stacks	shared/

2. Estructura del repositorio BQN-UY/CI-CD

GitHub Actions requiere que cada action tenga su propio `action.yml` en la raíz de su carpeta. La estructura usa dos niveles de agrupación: **capa** (shared / frontend / backend) y **stack** (python, scala, flutter, etc.).

```
BQN-UY/CI-CD
└── .github/
    └── actions/
        │
        ├── shared/                     # No sabe nada del lenguaje
        │   ├── security-scan/
        │   │   └── action.yml
        │   ├── semver-tag/
        │   │   └── action.yml
        │   ├── git-merge/
        │   │   └── action.yml
        │   ├── github-release/
        │   │   └── action.yml
        │   ├── deploy-trigger/
        │   │   └── action.yml
        │   └── label-check/
        │       └── action.yml
        │
        ├── frontend/
        │   ├── html-js/
        │   │   └── lint-build/         # ESLint + Prettier + build
        │   │       └── action.yml
        │   ├── vaadin/
        │   │   └── lint-build/         # Maven + Checkstyle + Java
        │   │       └── action.yml
        │   └── flutter/
        │       ├── lint-test/          # flutter analyze + test
        │       │   └── action.yml
        │       └── build-apk/          # apk / ipa / web
        │           └── action.yml
        │
        └── backend/
            ├── python/
            │   └── lint-test/          # Ruff + mypy + pytest
            │       └── action.yml
            ├── scala/
            │   └── lint-build/         # scalafmt + sbt compile/test
            │       └── action.yml
            └── node/
```

```
                 └── lint-test/         # ESLint + Jest/Vitest
                     └── action.yml
```

2.1 Regla de ubicación

Para decidir dónde va una action nueva, responder en orden:

1. ¿Menciona algún lenguaje, runtime o toolchain? Si no → `shared/`. Fin.
2. ¿Produce algo que el usuario final consume directamente? Sí → `frontend/`. No → `backend/`.
3. ¿Qué tecnología usa internamente? Esa es la subcarpeta: `python/`, `scala/`, `flutter/`, etc.

Ejemplo de decisión
security-scan → shared/ porque OpenGrep y BetterLeaks corren sobre cualquier repo sin importar el lenguaje.backend/python/lint-test → backend/ porque no produce UI, y python/ porque llama a pip, ruff y pytest.

2.2 Cómo se referencia desde un workflow

El path del `uses:` siempre sigue el mismo patrón:

```
# Shared — igual para cualquier repo
uses: BQN-UY/CI-CD/.github/actions/shared/security-scan@v2
uses: BQN-UY/CI-CD/.github/actions/shared/semver-tag@v2

# Frontend HTML+JS
uses: BQN-UY/CI-CD/.github/actions/frontend/html-js/lint-build@v2

# Backend Python
uses: BQN-UY/CI-CD/.github/actions/backend/python/lint-test@v2

# Backend Scala
uses: BQN-UY/CI-CD/.github/actions/backend/scala/lint-build@v2
```

3. Actions compartidas (shared/)

Las actions de `shared/` no contienen lógica de stack. Se usan igual desde cualquier tipo de repo y nunca deben modificarse al agregar un nuevo lenguaje.

3.1 shared/label-check

Verifica que el PR tenga exactamente uno de los labels obligatorios. Si no hay label, el CI falla y el merge queda bloqueado.

```
# .github/actions/shared/label-check/action.yml
name: Label check
description: Verifica label obligatorio en el PR

runs:
  using: composite
  steps:
    - name: Check PR label
      uses: mheap/github-action-required-labels@v5
      with:
        mode: exactly
        count: 1
        labels: |
          breaking-change
          feature
          fix
          chore
          deploy-action
      env:
        GITHUB_TOKEN: ${{ inputs.github-token }}

inputs:
  github-token:
    description: GITHUB_TOKEN del repo que invoca
    required: true
```

3.2 shared/security-scan

Corre OpenGrep (SAST) y BetterLeaks (secrets scanning) sobre el código. Si encuentra algo, el CI falla y el PR no puede mergearse.

```yaml
# .github/actions/shared/security-scan/action.yml
name: Security scan
description: OpenGrep (SAST) + BetterLeaks (secrets)

runs:
  using: composite
  steps:
    - name: OpenGrep SAST
      shell: bash
      run: |
        pip install opengrep --quiet
        opengrep scan --config auto --error \
          --output results-sast.sarif --sarif 2>/dev/null || true
        opengrep scan --config auto --error .

    - name: Upload SARIF
      uses: github/codeql-action/upload-sarif@v3
      with:
        sarif_file: results-sast.sarif
      continue-on-error: true

    - name: BetterLeaks
      shell: bash
      run: |
        pip install betterleaks --quiet
        betterleaks scan --fail-on-findings .
```

3.3 shared/semver-tag

Calcula el próximo tag SemVer 2.0 leyendo el historial de Git y lo crea firmado. El tipo de bump (major / minor / patch) lo elige el desarrollador en el `workflow_dispatch` de `make-release`.

```yaml
# .github/actions/shared/semver-tag/action.yml
name: SemVer tag
description: Calcula y crea tag SemVer desde historial Git

inputs:
  bump:
    description: major | minor | patch
    required: true
    default: minor
  prefix:
```

```
    description: Prefijo del tag
    default: v

outputs:
  version:
    description: Versión sin prefijo (ej. 1.3.0)
    value: ${{ steps.calc.outputs.version }}
  tag:
    description: Tag completo (ej. v1.3.0)
    value: ${{ steps.calc.outputs.tag }}

runs:
  using: composite
  steps:
    - name: Fetch tags
      shell: bash
      run: git fetch --tags --prune

    - name: Calculate version
      id: calc
      shell: bash
      run: |
        LATEST=$(git tag --list "${{ inputs.prefix }}*" \
          --sort=-version:refname | head -n1)
        LATEST=${LATEST:-${{ inputs.prefix }}0.0.0}
        VERSION=${LATEST#${{ inputs.prefix }}}
        IFS='.' read -r MAJOR MINOR PATCH <<< "$VERSION"
        case "${{ inputs.bump }}" in
          major) MAJOR=$((MAJOR+1)); MINOR=0; PATCH=0 ;;
          minor) MINOR=$((MINOR+1)); PATCH=0 ;;
          patch) PATCH=$((PATCH+1)) ;;
        esac
        NEW="$MAJOR.$MINOR.$PATCH"
        echo "version=$NEW" >> $GITHUB_OUTPUT
        echo "tag=${{ inputs.prefix }}$NEW" >> $GITHUB_OUTPUT

    - name: Create and push tag
      shell: bash
      run: |
        git config user.name  "github-actions[bot]"
        git config user.email
"github-actions[bot]@users.noreply.github.com"
        git tag -a "${{ steps.calc.outputs.tag }}" \
          -m "Release ${{ steps.calc.outputs.tag }}"
        git push origin "${{ steps.calc.outputs.tag }}"
```

3.4 shared/git-merge

Back-merge automático sin conflictos. Al no existir archivos de versión (no hay `version.sbt` ni equivalente), este paso siempre es limpio.

```
# .github/actions/shared/git-merge/action.yml
name: Git back-merge
description: Mergea rama origen a destino sin conflictos de versión

inputs:
  source:
    description: Rama de origen (ej. main)
    required: true
  target:
    description: Rama de destino (ej. develop)
    required: true

runs:
  using: composite
  steps:
    - name: Back-merge ${{ inputs.source }} → ${{ inputs.target }}
      shell: bash
      run: |
        git config user.name  "github-actions[bot]"
        git config user.email
"github-actions[bot]@users.noreply.github.com"
        git fetch origin
        git checkout ${{ inputs.target }}
        git merge origin/${{ inputs.source }} --no-edit \
          -m "chore: back-merge ${{ inputs.source }} → \
             ${{ inputs.target }} [skip ci]"
        git push origin ${{ inputs.target }}
```

3.5 shared/github-release

Crea un GitHub Release con release notes autogeneradas a partir de los PRs incluidos y sus labels.

```
# .github/actions/shared/github-release/action.yml
name: GitHub Release
description: Crea release con notas autogeneradas desde PRs
```

```
inputs:
  tag:
    description: Tag a publicar
    required: true

runs:
  using: composite
  steps:
    - name: Create GitHub Release
      uses: softprops/action-gh-release@v2
      with:
        tag_name: ${{ inputs.tag }}
        generate_release_notes: true
        make_latest: true
```

3.6 shared/deploy-trigger

Dispara el deploy vía webhook HTTP. El `service-url` y el `token` se configuran como secrets en cada repo del proyecto.

```
# .github/actions/shared/deploy-trigger/action.yml
name: Deploy trigger
description: Dispara deploy vía webhook configurable por repo

inputs:
  environment:
    description: dev | staging | production
    required: true
  service-url:
    description: URL del webhook de deploy
    required: true
  token:
    description: Token de autenticación
    required: true

runs:
  using: composite
  steps:
    - name: Trigger deploy
      shell: bash
      run: |
        HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
          -X POST "${{ inputs.service-url }}" \
```

```
          -H "Authorization: Bearer ${{ inputs.token }}" \
          -H "Content-Type: application/json" \
          -d '{
            "environment": "${{ inputs.environment }}",
            "ref": "${{ github.sha }}",
            "actor": "${{ github.actor }}"
          }')
        if [ "$HTTP_STATUS" != "200" ] && [ "$HTTP_STATUS" != "204" ]; then
          echo "Webhook failed — HTTP $HTTP_STATUS"
          exit 1
        fi
```

4. Caso 1 — Backend: Scala / Pekko REST API

4.1 Action específica de stack

backend/scala/lint-build/action.yml

Build completo con sbt: scalafmt check + compile + test. El versionado usa sbt-dynver — no hay `version.sbt`.

```yaml
# .github/actions/backend/scala/lint-build/action.yml
name: Scala lint & build
description: scalafmt + sbt compile + sbt test (sin version.sbt)

inputs:
  java-version:
    description: Versión de Java
    default: "21"
  scala-version:
    description: Versión de Scala
    default: "3.4.2"

runs:
  using: composite
  steps:
    - name: Setup Java
      uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: ${{ inputs.java-version }}
        cache: sbt

    - name: Setup sbt
      uses: sbt/setup-sbt@v1

    - name: Format check (scalafmt)
      shell: bash
      run: sbt scalafmtCheckAll

    - name: Compile
      shell: bash
      run: sbt compile

    - name: Test
      shell: bash
      run: sbt test
```

4.2 Workflows del proyecto Scala

ci.yml

```yaml
# BQN-UY/mi-api-scala/.github/workflows/ci.yml
name: CI

on:
  pull_request:
    branches: [develop]
  push:
    branches: ["release/**", "hotfix/**"]

jobs:
  verify-label:
    name: Verificar label de PR
    runs-on: ubuntu-latest
    if: github.event_name == "pull_request"
    steps:
      - uses: actions/checkout@v4
      - uses: BQN-UY/CI-CD/.github/actions/shared/label-check@v2
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}

  lint-build:
    name: Lint & Build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0       # sbt-dynver necesita historial Git
      - uses: BQN-UY/CI-CD/.github/actions/backend/scala/lint-build@v2

  security:
    name: Security scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0       # BetterLeaks analiza historial completo
      - uses: BQN-UY/CI-CD/.github/actions/shared/security-scan@v2
```

publish-snapshot.yml

En cada push a `develop` o `hotfix/*`, publica el JAR snapshot a Nexus. La versión la calcula sbt-dynver automáticamente (formato: `1.2.0+3-abc1234`).

```yaml
# BQN-UY/mi-api-scala/.github/workflows/publish-snapshot.yml
name: Publish snapshot

on:
  push:
    branches: [develop, "hotfix/**"]

jobs:
  publish:
    name: Publish JAR snapshot → Nexus
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: BQN-UY/CI-CD/.github/actions/backend/scala/lint-build@v2

      - name: Publish snapshot to Nexus
        shell: bash
        run: sbt publish
        env:
          NEXUS_USER:     ${{ secrets.NEXUS_USER }}
          NEXUS_PASSWORD: ${{ secrets.NEXUS_PASSWORD }}
          NEXUS_URL:      ${{ secrets.NEXUS_URL }}
```

start-release.yml

```yaml
# BQN-UY/mi-api-scala/.github/workflows/start-release.yml
name: Start release

on:
  workflow_dispatch:
    inputs:
      bump:
        description: Tipo de bump de versión
        required: true
        default: minor
```

```
        type: choice
        options: [major, minor, patch]

jobs:
  start-release:
    name: Crear release branch
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
          ref: develop

      - name: Calcular versión
        id: semver
        uses: BQN-UY/CI-CD/.github/actions/shared/semver-tag@v2
        with:
          bump: ${{ inputs.bump }}
          dry-run: true          # Solo calcula, no crea el tag

      - name: Crear release branch
        run: |
          git config user.name  "github-actions[bot]"
          git config user.email
"github-actions[bot]@users.noreply.github.com"
          git checkout -b "release/${{ steps.semver.outputs.tag }}"
          git push origin "release/${{ steps.semver.outputs.tag }}"
```

make-release.yml

```
# BQN-UY/mi-api-scala/.github/workflows/make-release.yml
name: Make release

on:
  workflow_dispatch:
    inputs:
      bump:
        description: Tipo de bump
        required: true
        default: minor
        type: choice
        options: [major, minor, patch]
      environment:
        description: Entorno de deploy
```

```yaml
        required: true
        default: production
        type: choice
        options: [staging, production]

jobs:
  release:
    name: Tag + Nexus + GitHub Release + deploy + back-merge
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: BQN-UY/CI-CD/.github/actions/backend/scala/lint-build@v2

      - uses: BQN-UY/CI-CD/.github/actions/shared/security-scan@v2

      - name: Crear tag SemVer
        id: tag
        uses: BQN-UY/CI-CD/.github/actions/shared/semver-tag@v2
        with:
          bump: ${{ inputs.bump }}

      - name: Publish release JAR → Nexus
        run: sbt publish
        env:
          NEXUS_USER:     ${{ secrets.NEXUS_USER }}
          NEXUS_PASSWORD: ${{ secrets.NEXUS_PASSWORD }}
          NEXUS_URL:      ${{ secrets.NEXUS_URL }}

      - uses: BQN-UY/CI-CD/.github/actions/shared/github-release@v2
        with:
          tag: ${{ steps.tag.outputs.tag }}
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

      - name: Merge release → main
        uses: BQN-UY/CI-CD/.github/actions/shared/git-merge@v2
        with:
          source: ${{ github.ref_name }}
          target: main

      - name: Back-merge main → develop
        uses: BQN-UY/CI-CD/.github/actions/shared/git-merge@v2
        with:
```

```
          source: main
          target: develop

      - uses: BQN-UY/CI-CD/.github/actions/shared/deploy-trigger@v2
        with:
          environment: ${{ inputs.environment }}
          service-url: ${{ secrets.DEPLOY_WEBHOOK_URL }}
          token:       ${{ secrets.DEPLOY_TOKEN }}
```

start-hotfix.yml

```
# BQN-UY/mi-api-scala/.github/workflows/start-hotfix.yml
name: Start hotfix

on:
  workflow_dispatch:
    inputs:
      description:
        description: Descripción corta del bug (ej. fix-null-parser)
        required: true

jobs:
  start-hotfix:
    name: Crear hotfix branch desde main
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
          ref: main

      - name: Calcular versión patch
        id: semver
        uses: BQN-UY/CI-CD/.github/actions/shared/semver-tag@v2
        with:
          bump: patch
          dry-run: true

      - name: Crear hotfix branch
        run: |
          git config user.name  "github-actions[bot]"
          git config user.email
"github-actions[bot]@users.noreply.github.com"
```

```
          BRANCH="hotfix/${{ steps.semver.outputs.tag }}-${{
inputs.description }}"
          git checkout -b "$BRANCH"
          git push origin "$BRANCH"
```

4.3 Versionado Scala — sbt-dynver

No existe `version.sbt`. El plugin sbt-dynver calcula la versión en build-time leyendo el historial de Git:

Situación	Rama	Versión calculada
Commit = tag v1.2.0	main	1.2.0
3 commits post v1.2.0	develop	1.2.0+3-abc1234
5 commits en hotfix	hotfix/v1.1.1	1.1.0+5-def5678
Árbol sucio (local)	cualquiera	1.2.0+3-abc1234+dirty

Sin conflictos en back-merge
Al no existir version.sbt, el back-merge automático de hotfix → develop siempre es limpio. Fue el problema histórico que este modelo resuelve.

5. Caso 2 — Frontend: HTML+JS · Backend: Python 3 + FastAPI

5.1 Actions específicas de stack

frontend/html-js/lint-build/action.yml

```yaml
# .github/actions/frontend/html-js/lint-build/action.yml
name: HTML+JS lint & build
description: ESLint + Prettier check + npm build

inputs:
  node-version:
    description: Versión de Node.js
    default: "20"

runs:
  using: composite
  steps:
    - name: Setup Node
      uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
        cache: npm

    - name: Install deps
      shell: bash
      run: npm ci

    - name: Lint (ESLint)
      shell: bash
      run: npx eslint . --max-warnings=0

    - name: Format check (Prettier)
      shell: bash
      run: npx prettier --check .

    - name: Build
      shell: bash
      run: npm run build    # genera /dist
```

backend/python/lint-test/action.yml

```yaml
# .github/actions/backend/python/lint-test/action.yml
name: Python lint & test
description: Ruff lint + format + mypy + pytest

inputs:
  python-version:
    description: Versión de Python
    default: "3.12"

runs:
  using: composite
  steps:
    - name: Setup Python
      uses: actions/setup-python@v5
      with:
        python-version: ${{ inputs.python-version }}
        cache: pip

    - name: Install deps
      shell: bash
      run: pip install -r requirements.txt -r requirements-dev.txt

    - name: Lint (Ruff)
      shell: bash
      run: ruff check .

    - name: Format check (Ruff format)
      shell: bash
      run: ruff format --check .

    - name: Type check (mypy)
      shell: bash
      run: mypy .

    - name: Tests (pytest)
      shell: bash
      run: pytest --tb=short -q
```

5.2 Workflows — Frontend HTML+JS

ci.yml

```yaml
# BQN-UY/mi-frontend/.github/workflows/ci.yml
```

```yaml
name: CI

on:
  pull_request:
    branches: [develop]
  push:
    branches: ["release/**", "hotfix/**"]

jobs:
  verify-label:
    name: Verificar label de PR
    runs-on: ubuntu-latest
    if: github.event_name == "pull_request"
    steps:
      - uses: actions/checkout@v4
      - uses: BQN-UY/CI-CD/.github/actions/shared/label-check@v2
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}

  lint-build:
    name: Lint & Build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: BQN-UY/CI-CD/.github/actions/frontend/html-js/lint-build@v2

  security:
    name: Security scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: BQN-UY/CI-CD/.github/actions/shared/security-scan@v2
```

publish-snapshot.yml — Frontend

Para frontend standalone el "snapshot" es el artefacto `/dist` guardado en GitHub Actions Artifacts. Si hay un CDN o storage propio, se agrega un paso de upload aquí.

```yaml
# BQN-UY/mi-frontend/.github/workflows/publish-snapshot.yml
name: Publish snapshot

on:
```

```
  push:
    branches: [develop, "hotfix/**"]

jobs:
  build-snapshot:
    name: Build & upload snapshot
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: BQN-UY/CI-CD/.github/actions/frontend/html-js/lint-build@v2

      - name: Upload /dist como snapshot
        uses: actions/upload-artifact@v4
        with:
          name: frontend-snapshot-${{ github.sha }}
          path: dist/
          retention-days: 14
```

make-release.yml — Frontend

```
# BQN-UY/mi-frontend/.github/workflows/make-release.yml
name: Make release

on:
  workflow_dispatch:
    inputs:
      bump:
        description: Tipo de bump
        required: true
        default: minor
        type: choice
        options: [major, minor, patch]
      environment:
        description: Entorno de deploy
        required: true
        default: production
        type: choice
        options: [staging, production]

jobs:
  release:
    name: Tag + GitHub Release + deploy + back-merge
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
```

```
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: BQN-UY/CI-CD/.github/actions/frontend/html-js/lint-build@v2

      - uses: BQN-UY/CI-CD/.github/actions/shared/security-scan@v2

      - name: Crear tag SemVer
        id: tag
        uses: BQN-UY/CI-CD/.github/actions/shared/semver-tag@v2
        with:
          bump: ${{ inputs.bump }}

      - uses: BQN-UY/CI-CD/.github/actions/shared/github-release@v2
        with:
          tag: ${{ steps.tag.outputs.tag }}
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

      - name: Merge release → main
        uses: BQN-UY/CI-CD/.github/actions/shared/git-merge@v2
        with:
          source: ${{ github.ref_name }}
          target: main

      - name: Back-merge main → develop
        uses: BQN-UY/CI-CD/.github/actions/shared/git-merge@v2
        with:
          source: main
          target: develop

      - uses: BQN-UY/CI-CD/.github/actions/shared/deploy-trigger@v2
        with:
          environment: ${{ inputs.environment }}
          service-url: ${{ secrets.DEPLOY_WEBHOOK_URL }}
          token:       ${{ secrets.DEPLOY_TOKEN }}
```

5.3 Workflows — Backend Python/FastAPI

ci.yml

```
# BQN-UY/mi-api-python/.github/workflows/ci.yml
```

```yaml
name: CI

on:
  pull_request:
    branches: [develop]
  push:
    branches: ["release/**", "hotfix/**"]

jobs:
  verify-label:
    name: Verificar label de PR
    runs-on: ubuntu-latest
    if: github.event_name == "pull_request"
    steps:
      - uses: actions/checkout@v4
      - uses: BQN-UY/CI-CD/.github/actions/shared/label-check@v2
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}

  lint-test:
    name: Lint & Test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: BQN-UY/CI-CD/.github/actions/backend/python/lint-test@v2

  security:
    name: Security scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: BQN-UY/CI-CD/.github/actions/shared/security-scan@v2
```

publish-snapshot.yml — Backend Python

Para backend Python el snapshot es una imagen Docker publicada en GHCR (GitHub Container Registry), equivalente al Nexus maven-snapshots del stack Scala.

```yaml
# BQN-UY/mi-api-python/.github/workflows/publish-snapshot.yml
name: Publish snapshot

on:
```

```
  push:
    branches: [develop, "hotfix/**"]

jobs:
  build-snapshot:
    name: Build & push Docker snapshot → GHCR
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: BQN-UY/CI-CD/.github/actions/backend/python/lint-test@v2

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build & push snapshot image
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: ghcr.io/bqn-uy/${{ github.event.repository.name }}:${{
github.sha }}
```

make-release.yml — Backend Python

```
# BQN-UY/mi-api-python/.github/workflows/make-release.yml
name: Make release

on:
  workflow_dispatch:
    inputs:
      bump:
        description: Tipo de bump
        required: true
        default: minor
        type: choice
        options: [major, minor, patch]
      environment:
        description: Entorno de deploy
```

```
        required: true
        default: production
        type: choice
        options: [staging, production]

jobs:
  release:
    name: Tag + Docker release + GitHub Release + deploy + back-merge
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    permissions:
      contents: write
      packages: write
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: BQN-UY/CI-CD/.github/actions/backend/python/lint-test@v2
      - uses: BQN-UY/CI-CD/.github/actions/shared/security-scan@v2

      - name: Crear tag SemVer
        id: tag
        uses: BQN-UY/CI-CD/.github/actions/shared/semver-tag@v2
        with:
          bump: ${{ inputs.bump }}

      - name: Login to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build & push release image
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: |
            ghcr.io/bqn-uy/${{ github.event.repository.name }}:${{
steps.tag.outputs.version }}
            ghcr.io/bqn-uy/${{ github.event.repository.name }}:latest

      - uses: BQN-UY/CI-CD/.github/actions/shared/github-release@v2
        with:
          tag: ${{ steps.tag.outputs.tag }}
        env:
```

```
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

      - name: Merge release → main
        uses: BQN-UY/CI-CD/.github/actions/shared/git-merge@v2
        with:
          source: ${{ github.ref_name }}
          target: main

      - name: Back-merge main → develop
        uses: BQN-UY/CI-CD/.github/actions/shared/git-merge@v2
        with:
          source: main
          target: develop

      - uses: BQN-UY/CI-CD/.github/actions/shared/deploy-trigger@v2
        with:
          environment: ${{ inputs.environment }}
          service-url: ${{ secrets.DEPLOY_WEBHOOK_URL }}
          token:       ${{ secrets.DEPLOY_TOKEN }}
```

6. Resumen comparativo de stacks

6.1 Action de stack por tipo de repo

Tipo de repo	Action de stack	Actions shared siempre presentes
Frontend HTML+JS	frontend/html-js/lint-build	label-check, security-scan, semver-tag, git-merge, github-release, deploy-trigger
Frontend Vaadin	frontend/vaadin/lint-build	ídem
Frontend Flutter	frontend/flutter/lint-testfrontend /flutter/build-apk	ídem
Backend Python/FastAPI	backend/python/lint-test	ídem
Backend Scala/Pekko	backend/scala/lint-build	ídem
Backend Node	backend/node/lint-test	ídem

6.2 Artefacto publicado por stack

Stack	Snapshot	Release
Scala/Pekko	JAR en Nexus maven-snapshots	JAR en Nexus maven-releases + GitHub Release
Python/FastAPI	Imagen Docker en GHCR (SHA tag)	Imagen Docker en GHCR (vX.Y.Z + latest) + GitHub Release
HTML+JS	Artefacto /dist en Actions	Artefacto /dist adjunto en GitHub Release
Vaadin	WAR/JAR en Nexus maven-snapshots	WAR/JAR en Nexus maven-releases + GitHub Release
Flutter	APK/IPA en Actions artifacts	APK/IPA adjunto en GitHub Release

6.3 Conventional Commits y labels

La misma convención aplica a todos los stacks:

Label PR	Tipo de commit	Bump SemVer sugerido
breaking-change	feat(scope)! o fix!	MAJOR (1.2.3 → 2.0.0)
feature	feat:	MINOR (1.2.3 → 1.3.0)
fix	fix:	PATCH (1.2.3 → 1.2.4)
chore	chore:, docs:, refactor:	PATCH

Label PR	Tipo de commit	Bump SemVer sugerido
deploy-action	Cualquiera + infra requerida	Independiente del bump

breaking-change vs deploy-action
breaking-change habla de compatibilidad de API — los consumidores deben actualizar su código. deploy-action habla de requisitos de infraestructura — el ambiente necesita preparación antes del deploy. Son dimensiones independientes y pueden coexistir en el mismo PR.

7. Cómo agregar un nuevo stack

La estructura está diseñada para escalar sin modificar nada existente. El procedimiento es siempre el mismo:

4. Decidir la capa: ¿frontend/ o backend/?
5. Crear la subcarpeta con el nombre del stack: `.github/actions/frontend/go/lint-build/` o `.github/actions/backend/go/lint-test/`.
6. Escribir el `action.yml` usando la plantilla de cualquier action existente del mismo tipo.
7. En el repo del proyecto, agregar el `uses:` correspondiente en los workflows. Las shared actions no cambian.
8. Abrir PR en BQN-UY/CI-CD con label feature. El CI valida que la nueva action funcione.

> **Zero impacto en repos existentes**
> Agregar backend/go/lint-test/ no toca ningún workflow de Python, Scala ni ningún otro stack. El principio de aislamiento está garantizado por la estructura de carpetas.

8. Archivos de soporte en cada repo de proyecto

8.1 pull_request_template.md

```
## Descripción
<!-- Qué hace este PR y por qué -->

## Tipo de cambio (seleccionar label en GitHub — obligatorio)
- [ ] `feature`        — nueva funcionalidad retrocompatible
- [ ] `fix`            — corrección de bug retrocompatible
- [ ] `breaking-change`— rompe compatibilidad de API
- [ ] `chore`          — refactor, deps, docs, tests
- [ ] `deploy-action`  — requiere acción previa en el ambiente

## Deploy action requerida
<!-- Si marcaste deploy-action, describir exactamente qué debe hacerse -->
<!-- Ej: migración DB, nueva variable de entorno, cambio de config -->
N/A

## Checklist
- [ ] Tests pasan localmente
- [ ] No hay secrets hardcodeados
- [ ] PR tiene exactamente un label asignado
```

8.2 labeler.yml

```
# .github/labeler.yml
breaking-change:
  - head-branch: [".*breaking.*", ".*BREAKING.*"]

feature:
  - head-branch: ["feature/.*"]

fix:
  - head-branch: ["fix/.*", "hotfix/.*"]

chore:
  - head-branch: ["chore/.*", "docs/.*", "refactor/.*"]
```

9. Roles y responsabilidades

Responsabilidad	Desarrollador	DevOps / Infra
Actions shared (CI-CD)	Propone cambios vía PR	Owner — aprueba y mergea
Actions de stack (CI-CD)	Owner — escribe y mantiene	Revisa
Workflows del proyecto	Owner	Revisa
Secrets del repo	—	Owner
Seguridad — hallazgos SAST	Resuelve	Asesora
Secretos detectados	Rota y limpia historial	Asesora
Decidir bump SemVer	Owner (en make-release)	—
Decidir breaking-change	Owner (en PR)	—
Decidir deploy-action	Owner (en PR)	Ejecuta en ambiente

BQN-UY · Equipo de Desarrollo · CI/CD v2 · Confidencial — uso interno · Marzo 2026